

Accelerating Super-Resolution CNN Inference via GPU Spatial Reduction: A Cross-Platform Study on Unified and Discrete Memory Architectures

M. Hamdani Ilham Latjoro, Adnan

Department of Informatics Engineering, Faculty of Engineering

Hasanuddin University

Makassar, Indonesia

latjoromhi25d@student.unhas.ac.id, adnan@unhas.ac.id

Abstract—Convolutional Neural Network (CNN) super-resolution models such as FSRCNN suffer from a severe computational bottleneck in their final deconvolution layer, where naive multi-threaded CPU accumulation causes data races and non-reproducible output corruption. We propose a GPU Spatial Reduction strategy that offloads FSRCNN’s Layer 8 deconvolution to isolated per-channel CUDA kernels followed by a parallel reduction pass, eliminating races by construction, and evaluate it on two architecturally distinct platforms: the NVIDIA GB10 Grace Blackwell Superchip (unified memory over NVLink-C2C) and a discrete-GPU desktop (Intel i9-14900K with an NVIDIA RTX 4090 over PCIe). On both platforms, V1 and V2 output is byte-for-byte identical to a verified deterministic reference at every configuration tested, and the GPU output is further bit-identical across the two platforms directly (0 differing bytes, matching SHA-256 digests) despite differing host ISA and GPU microarchitecture. Timings are reported as mean $\pm$ SD over the 6 measured runs of a 7-run repetition. The warp-aligned 32×8 CUDA grid is fastest at the kernel level on both GPU architectures, and per-kernel event profiling localizes the entire gain to the deconvolution kernel, exactly where the coalescing explanation predicts it. Against each platform’s best correctness-verified CPU configuration, GPU offloading achieves $1.46\times$ (GB10, 31.5 FPS) and up to $1.63\times$ (RTX 4090, 27–27.6 FPS) speedups; against the fastest bit-exact serial run the secondary figure is $3.84\times$ on the GB10. The determinism guarantee costs 4.6% of GPU Layer 8 time on the GB10 but only 0.6% on the RTX 4090, a gap traceable to whether the private buffer fits in last-level cache; conversely, device-to-host transfer is $4.74\times$ faster on unified memory. Against a constructed high-resolution reference the pipeline gains 2.17 dB PSNR-Y over bicubic upscaling. These results establish spatial reduction as a deterministic, cross-platform-portable pattern for GPU-accelerated CNN inference.

Index Terms—Super-resolution, FSRCNN, GPU offloading, CUDA, spatial reduction, Grace Blackwell, asymmetric multi-core, bit-exactness.

I. INTRODUCTION

Edge and workstation-class AI hardware increasingly adopt SoCs where multicore CPUs and GPUs co-exist on high-speed interconnects [3], [4]. Super-resolution CNNs progressed from early formulations [11] to FSRCNN [2], which achieves high quality by deferring feature extraction to low-resolution space and upscaling only at the final deconvolution layer.

That final stage (Layer 8) imposes a disproportionate computational burden: for $2\times$ upscaling ($176\times 144 \rightarrow 352\times 288$), it applies a 9×9 kernel across 56 channels and accumulates them into one output plane. Parallelizing this on multicore CPUs creates a dilemma: naive shared-memory accumulation causes data races and non-reproducible pixel corruption [1], while mutex/atomic synchronization causes prohibitive lock contention. The spatial reduction pattern avoids both by writing to private per-channel buffers and reducing them in a separate deterministic pass. That said, 56 private high-resolution buffers incur real memory overhead on CPU.

We offload this deconvolution and spatial reduction to CUDA kernels on an NVIDIA GPU while parallelizing Layers 1–7 across CPU cores, achieving high throughput, memory isolation, and deterministic bit-exactness. Our contributions:

- 1) GPU Cost-Profile Shift: privatization-reduction is a classical pattern, not a new algorithm; we show that moving it to the GPU changes its cost profile qualitatively. The 43.3 MiB of private-buffer traffic that costs 8–18% of CPU wall time is absorbed instead by GPU memory bandwidth and thread parallelism, while the result stays bit-exact with the CPU implementation. We frame this as a shift in bandwidth and parallelism rather than in memory capacity because our primary platform has no separate VRAM: the GB10’s 128 GB LPDDR5x pool is shared by CPU and GPU, so the buffer does not move anywhere, only the computation over it does.
- 2) Warp-Aligned Grid Configuration: a 32×8 thread block maximizes memory coalescing for the deconvolution’s row-major access, cutting Layer 8 kernel time 22.8% (GB10) and 17.7% (RTX 4090).
- 3) Rigorous Statistical Evaluation: on the GB10, GPU output is byte-for-byte identical to the CPU baseline (diff_bytes = 0) at every configuration tested, and timing measured over 7-run repetitions gives a $1.46\times$ speedup over the best deterministic CPU configuration (20 threads).
- 4) Cross-Platform Validation: an independent replication on discrete-PCIe hardware (Intel i9-14900K + RTX 4090)

confirms bit-exactness and warp-alignment generalize beyond the GB10, and GPU V2’s output is bit-identical (0 differing bytes, matching SHA-256) across both platforms directly, not just within each; this also surfaces a CPU thread-scheduling pathology on heterogeneous P/E-core silicon (Section VI).

II. BACKGROUND AND BASELINE ARCHITECTURE

A. Related Work

CNN super-resolution progressed from early formulations [11] to FSRCNN [2], increasingly deployed on heterogeneous CPU+GPU SoCs [3], [4]. Closest to ours, Annisa et al. [1] empirically characterize the same FSRCNN Layer 8 race condition on an Orange Pi 5 (Rockchip RK3588S: $4\times$ Cortex-A76 big + $4\times$ Cortex-A55 LITTLE, the same Suzie $176\times 144 \rightarrow 352\times 288$ sequence we use), finding race-induced corruption more severe on LITTLE-CPU affinity configurations than on big-CPU ones; their focus is on characterizing that corruption across core-affinity settings rather than on an algorithmic remedy for it. We do not compare speedup figures directly, since our hardware (GB10/RTX 4090) differs from theirs (Orange Pi 5) by orders of magnitude in core count and class. Privatize-then-reduce is a classical parallelization pattern rather than a new algorithm; our contribution is adopting it as a race-free CPU baseline (V1) and then moving it to the GPU (V2), which changes its cost profile qualitatively while staying bit-exact with the CPU reference. CNN inference on ARM big.LITTLE clusters scales well under proper scheduling [5], supporting our GB10 CPU baseline as non-trivial (Section V-A). General-purpose libraries like cuDNN [15] target convolution broadly but not this specific race-free layout (Section III-B); we do not claim to outperform them, only to control the memory layout. Discrete-GPU PCIe transfer overhead is well documented [9], motivating our unified-vs-discrete memory comparison (Section VI); recent work characterizes unified physical memory for CPU-GPU accelerators directly [21]. Our RTX 4090 thread-scheduling collapse (Section VI) relates to prior work on asymmetric-CPU scheduling [6], [7]; our contribution is an empirical observation, not a new mechanism.

B. FSRCNN Architecture

FSRCNN consists of eight layers: Feature Extraction (Layer 1), Shrinking (Layer 2), four Mapping layers (Layers 3–6), Expanding (Layer 7), and Deconvolution (Layer 8). For an input sequence at 176×144 scaled by $2\times$, Layers 1–7 operate in low-resolution (176×144) space, outputting 56 feature maps. Layer 8 receives these 56 channels, applies a 9×9 transposed convolution [16] with stride 2, and reduces them into a 352×288 YUV luminance output frame.

C. Baseline Variants

We analyze three distinct algorithmic variants:

- V0 (Naive OpenMP CPU): Implemented in `fsrcnn_parallel.c`. Layer 8 worker threads accumulate deconvolution outputs directly into a

shared single output buffer `img_filt_8`. When executed with threads > 1 , overlapping kernel footprints cause unsynchronized read-modify-write data races, leading to output corruption, the class of bug that dynamic race detectors such as ThreadSanitizer [13] are designed to catch.

- V1 (Spatial Reduction CPU): Implemented in `fsrcnn_parallel_spatial_reduction.c`. Eliminates data races by allocating a private intermediate buffer `all_tmp[ $56 \times 352 \times 288 \times 8$  B]` (43.3 MiB). Each channel writes to its own isolated offset. A second pass then sums the 56 channels per pixel. Both passes are parallel, but along different axes: the deconvolution is threaded over channels, the reduction over output pixels. The reduction needs no synchronization because each output pixel is written by exactly one thread, and it stays deterministic because that thread accumulates its 56 summands in fixed channel order regardless of how the loop is scheduled.
- V2 (GPU Spatial Reduction - Proposed): Implemented in `fsrcnn_gpu_main.cu` and `fsrcnn_gpu.cu`. Layers 1–7 are parallelized across CPU cores using OpenMP, while Layer 8 deconvolution and channel reduction are offloaded to CUDA kernels on the NVIDIA Blackwell GPU.

III. GPU SPATIAL REDUCTION IMPLEMENTATION

A. Hardware Targets: GB10 and RTX 4090

Our primary experimental platform is the ASUS Ascent GX10 AI Supercomputer, equipped with the NVIDIA GB10 Grace Blackwell Superchip [17], whose CPU cluster is built on Arm’s DynamIQ big.LITTLE cluster technology [18]. Section VI cross-validates our approach on a second, architecturally distinct platform: an Intel Core i9-14900K desktop with an NVIDIA GeForce RTX 4090 (Ada Lovelace [19]), where CPU and GPU memory are physically separate and PCIe-connected rather than sharing the GB10’s unified NVLink-C2C pool. Table I compares both platforms.

TABLE I
HARDWARE SPECIFICATIONS: GB10 (GX10) VS. RTX 4090 DESKTOP

Component	GB10 (ASUS GX10)	RTX 4090 Desktop
CPU	20-core ARM v9.2-A DynamIQ 10 $\times$ X925 P + 10 $\times$ A725 E @ 3.90/2.81 GHz, no SMT	Intel Core i9-14900K 8 P-core (w/HT) + 16 E-core 24C/32T, up to 6.0 GHz
GPU	NVIDIA Blackwell, <code>sm_121</code>	NVIDIA Ada Lovelace, <code>sm_89</code>
GPU Cores	6,144 CUDA, 48 SMs, Gen5 Tensor	16,384 CUDA, 128 SMs [19]
Memory	128 GB LPDDR5x, <i>unified</i>	64 GB DDR (63 GB OS-visible) + 24 GB GDDR6X, <i>discrete</i>
Interconnect	NVLink-C2C (5 $\times$ PCIe 5.0)	PCIe (CPU $\leftrightarrow$ GPU)
CUDA	13.0, Driver 580.159.03, V13.0.88	13.0, Driver 580.65.06, V13.0.48
GCC	13.3.0 (Ubuntu 24.04)	13.3.0 (Ubuntu 24.04)
OS	Ubuntu 24.04 LTS (narch64)	Ubuntu 24.04 LTS (x86_64)

B. CUDA Kernel Design and Warp Alignment

We implement Layer 8 as two specialized hand-written CUDA kernels rather than using a general-purpose library such as cuDNN [15]. We do not claim this is faster than a cuDNN-based deconvolution (untested, and a well-tuned library kernel could plausibly win); we only claim that it gives direct control

over the private-buffer-plus-reduction layout central to our correctness argument. A head-to-head comparison against cuDNN and simpler alternatives (atomicAdd, output-stationary gather) is future work (Section VII); our result shows this design is faster and race-free relative to the CPU baseline, not that it is the fastest possible GPU design.

1) *Deconvolution Kernel (deconv_kernel)*: Each CUDA thread block processes a 2D tile of the input feature map. For each channel $c \in [0, 55]$, threads compute the 9×9 transposed convolution and store the result directly into an isolated channel plane inside GPU VRAM buffer `d_all_tmp`, indexed as $\text{index}(c, y, x) = c \times (H_{\text{out}} \times W_{\text{out}}) + y \times W_{\text{out}} + x$.

2) *Spatial Reduction Kernel (spatial_reduction_kernel)*: After deconvolution completes, the spatial reduction kernel assigns one CUDA thread per high-resolution output pixel (y, x) . The thread iterates across all 56 channel planes in `d_all_tmp`, sums the values, and adds the Layer 8 bias:

$$\text{output}[y, x] = \left(\sum_{c=0}^{55} \text{d_all_tmp}[c, y, x] \right) + \text{bias}_8 \quad (1)$$

Because channel writes are fully isolated during deconvolution and read-only during reduction, the execution is completely race-free and deterministic.

NVIDIA GPUs execute threads in SIMD groups of 32 called *warps* [8]. Standard 16×16 blocks place only 16 threads along the row-major X dimension, causing consecutive warps to straddle non-contiguous memory rows. Since warp/block sizing has a first-order effect on SIMT throughput [22], we evaluate a 32×8 block (256 threads): setting $B_x = 32$ guarantees all 32 threads in a warp access contiguous doubles along one image row, maximizing coalescing.

IV. EXPERIMENTAL EVALUATION

A. Evaluation Methodology

Experiments were conducted on the ASUS Ascent GX10 under Ubuntu 24.04 LTS ARM64. The workload consists of 150 frames of the Suzie video sequence ($176 \times 144 \rightarrow 352 \times 288$, YUV420p).

To guarantee statistical rigor, every configuration was executed for 7 repetitions: run 1 discarded as cold-start/warmup, runs 2–7 (6 runs) recorded as Mean $\pm$ SD.

Reproducibility: CPU binaries are compiled with `gcc -fopenmp -O3 (no -march=native/-ffast-math)`; parallel loops use plain `#pragma omp parallel for` with no explicit `schedule()`, so thread placement follows the OpenMP runtime’s default (GNU libgomp: static). The GPU binary is compiled with `nvcc -O3 -std=c++11 -arch=native`, which resolves to `sm_121` on the GB10 and `sm_89` on the RTX 4090. GCC 13.3.0 (Ubuntu 24.04) was used on both platforms, and both use CUDA release 13.0, differing only in patch level (V13.0.88 on the GB10 vs. V13.0.48 on the RTX 4090; Table I). All V1/V2 arithmetic uses double precision (`double`), matching the CPU baseline and making the bit-exact CPU $\equiv$ GPU comparison below as strict as possible; FP32

would admit more CPU/GPU rounding divergence even for a logically-equivalent computation (RTX-specific throughput consequence in Section VI).

Reference file: We generate one verified deterministic reference, `ground_truth.yuv`, by running V1 (race-free by construction) single-threaded once; since V0, V1, and V2 implement the identical FSRCNN algorithm and differ only in execution strategy, this file is an internal determinism check, not an independently sourced high-resolution ground truth. Correctness is assessed two ways: a direct byte comparison (`cmp -l`) between V1 and V2 output (`diff_bytes = 0` across all grid configurations), and scoring each variant against this reference via byte diff, PSNR, and SSIM [10] (`ffmpeg`). V1/V2’s PSNR = ∞ /SSIM = 1.0 against it is thus logically equivalent to the `cmp -l` result, not a separate quality measurement; we report it as *bit-exact* rather than a numeric figure (Tables II, IV). This confirms cross-variant determinism, not how close FSRCNN’s reconstruction is to the true high-resolution Suzie sequence; Section IV-C addresses that separately with a constructed high-resolution reference.

Unless stated otherwise, PSNR/SSIM figures in Tables II and IV are `ffmpeg`’s all-component average over Y, U and V. Because only the Y plane passes through the network, the untouched chroma planes score identically to the reference and pull the average upward: the RTX 4090’s worst V0 case reads 23.0 dB/0.845 all-component but 21.5 dB/0.767 on luminance alone. The all-component figures therefore *understate* the corruption, which does not weaken the determinism argument, but we give luminance-only numbers wherever the magnitude of the degradation is what matters (Fig. 1, Section IV-C).

V0, in contrast to V1/V2, shows real, non-monotonic degradation from its data race against the same file: as low as 13.8 dB/0.518 on the GB10 (4 threads), and, more strikingly, 68.6 dB/0.99996 on the RTX 4090 (4 threads) before collapsing to 23.0 dB/0.845 at 32 threads. The RTX case matters more: a race degrading output by only 68.6 dB is invisible to casual inspection or a coarse PSNR threshold, yet exactly as non-deterministic as the 13.8 dB case, an argument for determinism-by-construction over statistical trust that runs “look fine.” The non-determinism is directly observable: three consecutive V0 runs at 32 threads on the RTX 4090 corrupted 4.75, 5.01 and 5.04 million of 22.8 million output bytes, scoring 21.34, 21.46 and 21.63 dB PSNR-Y — never the same output twice. Fig. 1 shows this directly (reference, V1, V2 visually identical; V0 shows clear scanline corruption from unsynchronized writes to `img_flt_r8`).

B. Performance Scaling & Accuracy Benchmark

Table II presents the complete experimental results comparing CPU V0 (Naive), CPU V1 (Spatial Reduction), and GPU V2 (Proposed Hybrid) across thread counts, PSNR, SSIM structural quality, and GPU grid configurations.

TABLE II

KEY CONFIGURATIONS: PERFORMANCE AND DEVIATION FROM THE VERIFIED DETERMINISTIC REFERENCE, 7-RUN REPETITIONS (MEAN $\pm$ SD OVER 6 MEASURED RUNS). FULL CPU THREAD SWEEP IN FIG. 2(A).

Variant	Configuration / Threads	Wall Time (ms)	Speedup vs. Serial	Speedup vs. Best CPU	Dev. from Ref. (dB)	SSIM	GPU L8 Kernel (ms)
CPU V0 (Naive)	1 Thread	18,240.50 $\pm$ 228.82	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (worst)	7,074.33 $\pm$ 90.21	2.58 $\times$	N/A	13.785	0.518246	N/A
CPU V0 (Naive)	20 Threads (Max, fastest)	6,346.33 $\pm$ 32.60	2.87 $\times$	N/A	25.751	0.838930	N/A
CPU V1 (Baseline)	1 Thread (Serial)	21,552.33 $\pm$ 298.07	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1	4 Threads (cf. RTX optimum)	8,316.50 $\pm$ 58.06	2.59 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1 (Max Cores)	20 Threads (P+E cores)	6,921.83 $\pm$ 82.49	3.11 $\times$	N/A	bit-exact	bit-exact	N/A
GPU V2 (Hybrid)	Grid 16 $\times$ 16_256 (baseline)	4,935.67 $\pm$ 36.30	4.37 $\times$	1.40 $\times$	bit-exact	bit-exact	696.51 $\pm$ 1.78
GPU V2 (Best Grid)	Grid 32 $\times$ 8_256	4,756.33 $\pm$ 30.71	4.53 $\times$	1.46 $\times$	bit-exact	bit-exact	533.19 $\pm$ 2.58

Speedup vs. Serial: CPU rows against their own variant’s 1-thread time; GPU V2 has no 1-thread time of its own, so its denominator is CPU V1’s serial time (21,552.33 ms). Section V-A also reports V2 against the faster bit-exact V0 serial time. Speedup vs. Best CPU: filled only for GPU V2, against CPU V1’s 20-thread configuration (fastest correctness-verified CPU result, Section V-A); N/A elsewhere since it is not the quantity those rows measure. GPU V2 runs Layers 1–7 on 20 OpenMP CPU threads before offloading Layer 8; remaining 3 grid configs (8 $\times$ 8_256, 16 $\times$ 16_128, 16 $\times$ 16_512) in Fig. 3.

Deviation/SSIM vs. the verified deterministic reference (Section IV-A); “bit-exact” rows are logically equivalent to the `cmp -1` result, not an independent quality measurement (one measurement per configuration, not per-run). V0 shows real, non-monotonic degradation from its data race (worst at 4 threads: 13.8 dB/0.518). GPU L8 Kernel times are mean $\pm$ SD over 6 runs of the dedicated profiling sweep (Section IV-E), captured separately from the wall-time sweep in this table.

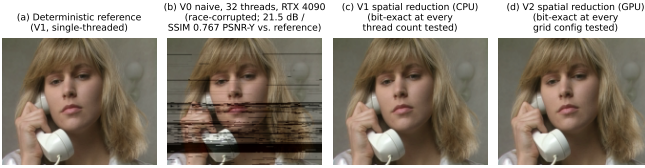


Fig. 1. Frame 26 of the reconstructed Suzie sequence across all four output variants. (a) Verified deterministic reference (V1, single-threaded). (b) V0’s naive accumulation at 32 threads on the RTX 4090 (Intel i9-14900K host), visibly corrupted by unsynchronized concurrent writes to `img_flt_r_8` (horizontal scanline artifacts); this is one representative run of three, which scored 21.34–21.63 dB PSNR-Y and 0.767–0.779 SSIM-Y against (a) and never reproduced the same output twice. (c) V1 (CPU spatial reduction) and (d) V2 (GPU spatial reduction) are both pixel-identical to (a) at every thread count and grid configuration tested, visually indistinguishable from the reference unlike (b).

C. Reconstruction Quality Against a True High-Resolution Reference

Every correctness figure reported so far is a self-consistency check: V2 against V1, which is why it reads “bit-exact” rather than a PSNR value. That establishes the GPU port is faithful to the CPU algorithm, but it says nothing about whether FSRCNN reconstructs anything, because no high-resolution ground truth exists anywhere in the pipeline — `suzie_qcif.yuv` is the input, not a downsampled version of something larger.

We therefore constructed the missing reference. Suzie was bicubic-downsampled from 176 $\times$ 144 to 88 $\times$ 72, that sequence was fed to the same FSRCNN pipeline at scale 2 to reconstruct 176 $\times$ 144, and the result was scored against the original 176 $\times$ 144 frames. Bicubic upscaling of the same 88 $\times$ 72 input is the comparator: it is what the network has to beat to justify its runtime. We report luminance-only figures, since only the Y plane passes through the network (chroma is 2 $\times$ pixel replication in this implementation), and an all-component average would credit FSRCNN for a replication step it does not perform.

FSRCNN reaches 37.86 dB PSNR-Y and 0.9665 SSIM-Y, against bicubic’s 35.69 dB and 0.9532: a gain of 2.17 dB and 0.0133 SSIM over the whole 150-frame sequence. The network earns its cost. Two caveats belong with that number. It is measured on one sequence at one scale factor, so it is a sanity check on the pipeline rather than a claim about FSRCNN’s quality in general, which is established in the original work [2]. And on the chroma planes bicubic is in fact *better* (50.45/49.71 dB against 48.56/47.48 dB), precisely because it interpolates U and V while our pipeline replicates them — a limitation of this implementation, not of spatial reduction, and one that would be worth fixing before deploying the pipeline for perceptual quality rather than throughput measurement.

D. The Cost of Determinism, Quantified

Contribution 1 (Section I) claims that privatization-reduction is expensive on the CPU and absorbed by the GPU. Tables II and IV let us quantify the CPU side directly as the wall-time delta between V0 (racy, no privatization) and V1 (privatized, deterministic) at matched thread counts. On the GB10: +18.2% at 1 thread (18,240.50 $\rightarrow$ 21,552.33 ms), +17.6% at 4 threads (7,074.33 $\rightarrow$ 8,316.50 ms), +9.1% at 20 threads (6,346.33 $\rightarrow$ 6,921.83 ms). On the RTX 4090: +9.2% at 1 thread (20,973.50 $\rightarrow$ 22,904.17 ms), +7.8% at 4 threads (8,207.67 $\rightarrow$ 8,844.17 ms). This delta isolates privatization specifically rather than a difference in how much of the work is threaded: V0 and V1 parallelize the same deconvolution, and V1’s additional reduction pass is itself parallel over output pixels (Section II-C), so the gap measures the cost of writing and re-reading 43.3 MiB of private buffers, not the cost of serializing a stage. Determinism therefore costs 8–18% of CPU wall time. The GB10 points suggest the cost falls once the machine is saturated, but 1 thread and 4 threads are nearly identical there (18.2% vs. 17.6%), so this is a two-level effect at best, not a smooth trend.

On the GPU side the same 43.3 MiB of private-buffer traffic carries no measurable wall-time penalty: the 32×8 kernel still wins outright (Section V-A) with peak device memory bounded at 44.3 MiB including allocator padding, well under 0.1% of the 128 GB the GB10’s CPU and GPU share or of the RTX 4090’s 24 GB of dedicated VRAM. We state this as a bandwidth-and-parallelism result rather than a capacity result, because on the GB10 there is no separate VRAM for the buffer to move into.

Per-kernel event instrumentation (Section IV-E) now lets us put the GPU-side cost in the same units as the CPU-side 8–18%. The reduction pass is the GPU analogue of the privatization overhead, and it accounts for 4.6% of the Layer 8 event window on the GB10 and 0.6% on the RTX 4090 — in both cases well below what the same determinism guarantee costs on the CPU, and on the discrete GPU almost free. The reason the two platforms differ so much is architectural rather than algorithmic: the reduction is bandwidth-bound, and the RTX 4090’s L2 is large enough to hold the entire private buffer (Section IV-E). Determinism-by-construction is therefore cheapest exactly where cache capacity exceeds the privatization working set, which is a useful design rule for anyone applying this pattern to a larger network: the overhead scales with how far the private buffer overflows the last-level cache, not with the channel count as such.

E. Fine-Grained GPU Timing Breakdown

To evaluate each stage’s contribution we instrumented CUDA Event profiling inside `fsrcnn_gpu_main.cu`, and ran the optimal configuration ($32 \times 8_{256}$) six times on each platform. Table III reports the result for both.

A correction to how we previously read this instrumentation. An earlier draft of this work treated the host-side counter `h2d_ms` as a disjoint slice of wall time and reported host-to-device transfer as 516.74 ms, roughly 11% of the pipeline. That reading was wrong, and the six-run sweep makes the error visible. The counter accumulates host wall time spent inside `cudaMemcpy`, but those copies are issued *inside* the CUDA-event window that `gpu_18_ms` measures, and a blocking copy on the default stream cannot begin until the previously launched kernel has drained. `h2d_ms` therefore charges kernel-completion waiting to transfer, and double-counts time already inside `gpu_18_ms`. The decisive evidence is that `h2d_ms` tracks `gpu_18_ms` at a nearly fixed ratio across all five grid configurations (0.96–0.98 on the GB10, 0.88–0.90 on the RTX 4090) and moves by 167 ms on the GB10 when the *kernel block shape* alone changes, even though the transferred byte count is identical in every configuration. No genuine transfer measurement behaves that way. We report the corrected decomposition below and no longer make the “H2D asymmetry” claim that rested on the old reading (Section V-B).

The corrected split comes from bracketing each `deconv_kernel` launch and the `spatial_reduction_kernel` with their own CUDA events, which also resolves the separation Section IV-D needed. Residual in-window time (real transfer, launch gaps,

and host-side padding between launches) is reported as its own line rather than folded into either kernel.

Two results stand out, and they point in opposite directions.

First, the reduction kernel costs 26.32 ms on the GB10 but only 2.62 ms on the RTX 4090, a $10.0 \times$ gap far larger than any general compute advantage (`deconv_kernel` differs by only $1.6 \times$). The reduction is purely bandwidth-bound: it reads the whole 43.31 MiB private buffer once per frame, 6.81 GB over 150 frames. That works out to 259 GB/s on the GB10, about 95% of the 273 GB/s theoretical peak of its LPDDR5x pool, and to 2,597 GB/s on the RTX 4090, which *exceeds* that card’s 1,008 GB/s of GDDR6X bandwidth by more than $2.5 \times$. A DRAM-resident working set cannot produce that number. The natural explanation is the RTX 4090’s 72 MB L2 cache: the 43.31 MiB buffer fits inside it, so the reduction is served largely from cache, whereas the GB10 must stream the same buffer from its unified memory pool at close to hardware peak. We offer this as the reading most consistent with the measured bandwidths rather than a profiled fact, since we did not capture L2 hit-rate counters (Section VII).

Second, device-to-host transfer runs the other way: 3.46 ms on the GB10 against 16.42 ms on the RTX 4090, a $4.74 \times$ advantage for unified memory on identical payloads (35.1 vs. 7.4 GB/s effective). This is the cleanest unified-versus-discrete contrast in our data, and unlike the retracted H2D figure it is measured outside the event window, after `cudaEventSynchronize`, with no kernel execution to conflate it with.

Peak device memory stayed bounded at 45,382 KiB (≈ 44.3 MiB) on both platforms across every GPU configuration, about 1 MiB above the 43.3 MiB theoretical minimum (Section II-C), consistent with modest allocator padding.

V. DISCUSSION AND KEY INSIGHTS

A. Speedup vs. Maxed-Out CPU Cores, and Warp Alignment

We report GPU speedup against CPU V1, not V0, at multi-thread configurations: V1 is race-free by construction at every thread count, while V0 is only *incidentally* correct at 1 thread (no concurrency to race on) and degrades with more threads (Table II). V0 is in fact the raw-wall-time winner at 20 threads (6,346.33 ms, faster than V1 and giving only $1.33 \times$ GPU speedup if used), but we exclude it as a baseline since its output there is not verified correct; we state the number here rather than omitting it.

GPU V2 ($32 \times 8_{256}$) reaches 4,756.33 ms end-to-end (31.5 FPS). Against CPU V1’s 20-thread configuration (6,921.83 ms, the fastest correctness-verified CPU result) this is $1.46 \times$ (46% faster), our primary headline figure, consistent with how we report the RTX 4090 (Section VI).

The serial comparison needs more care, because the two variants do not share a serial baseline. At 1 thread V0 is bit-exact (Table II) and also faster than V1 (18,240.50 vs. 21,552.33 ms), since it skips the privatization V1 pays for. V0 at 1 thread is therefore the fastest *bit-exact* serial baseline available, and the defensible serial figure is the one measured against it: $3.84 \times$ on the GB10, and $3.86 \times$ on the RTX

TABLE III
FINE-GRAINED PROFILING BREAKDOWN, OPTIMAL CONFIGURATION (32×8_256, 150 FRAMES), MEAN ± SD OVER 6 RUNS

Pipeline Stage	GB10 (unified)		RTX 4090 (discrete)	
	Duration (ms)	%	Duration (ms)	%
CPU Layers 1–7 (<code>cpu_117_ms</code>)	3,392.82 ± 21.62	71.4%	4,624.50 ± 70.96	84.5%
GPU Layer 8, event window (<code>gpu_18_ms</code>)	533.19 ± 2.58	11.2%	360.92 ± 7.22	6.6%
↪ <code>deconv_kernel</code> (56×)	424.07 ± 0.76	73.4% [†]	261.90 ± 3.09	62.9% [†]
↪ <code>spatial_reduction_kernel</code> (1×)	26.32 ± 1.02	4.6% [†]	2.62 ± 0.01	0.6% [†]
↪ in-window transfer and launch gaps	127.65 ± 4.10	22.1% [†]	152.14 ± 0.36	36.5% [†]
Device-to-Host (<code>d2h_ms</code>)	3.46 ± 0.06	0.1%	16.42 ± 0.49	0.3%
Unaccounted (process launch, file I/O)	821.20	17.3%	469.33	8.6%
Total Pipeline Wall Time	4,750.67 ± 30.40	100.0%	5,471.17 ± 76.17	100.0%

[†]Indented rows are a decomposition of the `gpu_18_ms` window directly above them, and their percentages are shares of *that window*, not of wall time; they are not additional wall-time slices. They come from a separately instrumented binary whose per-launch event recording inflates the window itself by 8.4% (GB10) and 15.4% (RTX 4090) relative to the production binary, so we report them as proportions rather than absolute times. The production `h2d_ms` counter is deliberately omitted: it overlaps `gpu_18_ms` and is not a transfer measurement (see text).

4090 (20,973.50/5,435.50). Measuring against V1’s serial time instead yields the larger 4.53× and 4.21×; that choice keeps the denominator inside one variant family, but the slower denominator is what makes the ratio bigger, so we lead with 3.84× and report 4.53× only as the within-variant figure.

This 1.46× is notable given CNN inference scales well on ARM big.LITTLE clusters when properly scheduled [5]; the GB10 CPU baseline is not a strawman. GPU V2 also has lower variance (±30.71 ms) than CPU V1 at its own best configuration (20 threads, ±82.49 ms, 2.7× higher) or at its noisiest (8 threads, ±998.43 ms). Figure 2(a) plots both CPU variants across the full thread ladder: unlike the RTX 4090 (Fig. 2(b)), the GB10’s 20 homogeneous cores (no SMT) scale smoothly with no collapse.

Shifting from the default 16×16 grid (696.51 ± 1.78 ms kernel time, 6 runs) to the warp-aligned 32×8 grid (533.19 ± 2.58 ms) reduces Layer 8 kernel time by 163.32 ms (23.4%). The separation exceeds fifty standard deviations, so unlike the wall-clock comparison this is not a marginal effect.

We attribute it to memory coalescing: a 32-thread-wide block lets every thread in a warp issue a request into the same coalesced transaction, rather than splitting it across multiple transactions as a 16-wide block does. The per-kernel breakdown supports this more sharply than wall time can. Block shape is a property of `deconv_kernel` only — `spatial_reduction_kernel` is launched one-dimensionally and is unaffected by the grid parameter — so if coalescing is the mechanism, the entire gain must appear in the deconvolution and none in the reduction. That is what we measure: going from 16×16 to 32×8, `deconv_kernel` drops 24.9% (564.74 ± 0.75 to 424.07 ± 0.76 ms) while `spatial_reduction_kernel` moves by +0.9% (26.08 to 26.32 ms), within its own run-to-run spread. The improvement is localized exactly where the mechanism predicts. This remains an inference from timing rather than a direct measurement of load efficiency (e.g. via `ncu` counters), and a sweep including non-aligned widths would strengthen it further (Section VII).

B. Transfer Cost and Amdahl’s Law

With the measurement error in Section IV-E corrected, transfer is not the bottleneck we previously took it for. On the GB10, everything inside the event window that is not kernel execution — real host-to-device traffic, launch gaps, and the host-side padding done between launches — totals 127.65 ms, or 2.7% of wall time, and device-to-host adds a further 3.46 ms (0.023 ms/frame). Even treating that entire residual as pure transfer, which overstates it, moves under 3% of the pipeline. The corresponding residual on the RTX 4090 is larger in both absolute and relative terms (152.14 ms, 2.8% of a longer wall time, and 36.5% of its event window against the GB10’s 22.1%), which is the direction PCIe versus NVLink-C2C would predict, though the margin is small enough that we do not lean on it. The one transfer measurement that is both clean and clearly asymmetric is D2H, where unified memory is 4.74× faster (Section IV-E).

What Table III shows instead is a straightforward Amdahl’s Law [12] ceiling. Offloading Layer 8 shrank it to 533.19 ms (3.55 ms/frame), 11.2% of wall time on the GB10 and just 6.6% on the RTX 4090, while CPU Layers 1–7 — still running on 20 and 4 OpenMP threads respectively, not serial — now account for 71.4% and 84.5%. The accelerated stage has been optimized into near-irrelevance, and further tuning of Layer 8 cannot buy much: even making it free would leave the RTX 4090 pipeline at 93% of its current wall time. Offloading Layers 1–7 is therefore the only remaining path to sub-second reconstruction, and a Roofline analysis [14] would clarify whether that offload is compute- or memory-bound first. The unaccounted remainder is process launch, file I/O, and the per-frame `uint8↔double` conversion loops, all of which sit outside the instrumented region and all of which are single-threaded. It is larger on the GB10 in both absolute and relative terms (821.20 ms, 17.3%, against 469.33 ms, 8.6%), which is consistent with that serial host code running on a 3.9 GHz ARM core rather than a 6.0 GHz x86 one; we did not instrument it separately to confirm.

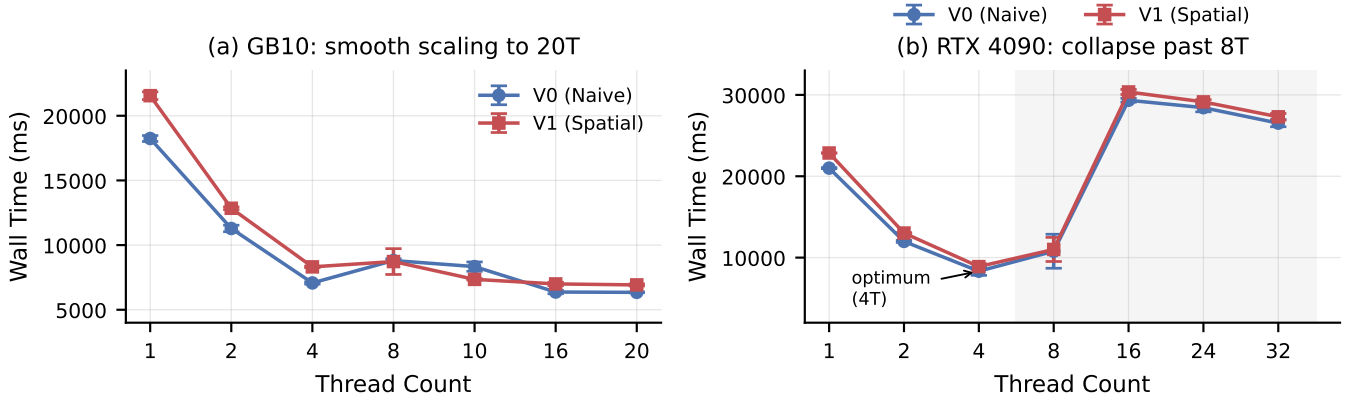


Fig. 2. CPU wall time vs. thread count, both platforms. (a) The GB10’s 20 homogeneous-per-tier cores (no SMT) scale smoothly to their maximum. (b) The RTX 4090’s heterogeneous P-core/E-core/Hyper-Threading CPU collapses past 8 threads (shaded), recovering only partially by 32.

VI. CROSS-PLATFORM VALIDATION ON DISCRETE-GPU HARDWARE

To test whether the GPU Spatial Reduction approach generalizes beyond the GB10’s unified-memory architecture, we replicated the experiment on a conventional discrete-GPU desktop (Intel Core i9-14900K + NVIDIA RTX 4090; specifications in Table I), where CPU (system DDR) and GPU (VRAM) memory are physically separate and connected over PCIe rather than NVLink-C2C. Unlike the GB10’s 20 homogeneous-per-tier ARM cores with no simultaneous multithreading (SMT), this CPU has a heterogeneous 8 Performance-core + 16 Efficiency-core layout where only the P-cores support Hyper-Threading, yielding 24 physical / 32 logical threads.

A. Platform-Specific Methodology Adjustment

A first attempt reusing the GB10’s thread ladder (1–20) and fixed 20-thread CPU backdrop produced misleading results here: wall time collapsed past 8 threads ($\sim 3.5\times$ slower than the 4-thread optimum), and since that degraded configuration backdropped every GPU grid-search run, results were dominated by CPU contention rather than the GPU kernel. We therefore (1) swept a topology-correct ladder (1, 2, 4, 8, 16, 24, 32, matching this chip’s core boundaries) and (2) used this platform’s own best CPU configuration (4 threads) as the backdrop, rather than assuming more threads is optimal. Table IV reports the corrected results.

B. The Collapse Is Reproducible, and GPU Speedup Against a Fair Baseline

The collapse was observed independently across *three* separate full runs (two on the 1–20 ladder, one on the corrected 1–32 ladder reported here), with tight SDs at each collapsed point (e.g. 16 threads: $29,341 \pm 240$ ms here vs. $29,511 \pm 125$ and $29,609 \pm 249$ ms previously), which rules out a one-off artifact. This chip’s P/E-core split relies on Intel Thread Director for scheduling, and asymmetric-CPU system software generally needs explicit heterogeneity awareness to avoid

throughput loss [6]; the generic OpenMP pool used here has no such affinity control [7] (OMP_PLACES/proc_bind), so past 8 threads the scheduler appears to actively hurt throughput. Peak performance is at exactly 4 threads, not the nominal 32-thread maximum. This optimum does not cleanly match the chip’s topology (8 P-cores, Table I), so we do not claim a confirmed topological explanation, and cannot rule out memory-bandwidth or DVFS/thermal effects without a control experiment (Section VII). Figure 2(b) plots this curve.

With the CPU backdrop corrected, GPU V2 reaches 5,435–5,579 ms end-to-end (26.9–27.6 FPS), a $1.59\times$ – $1.63\times$ speedup over CPU V1’s best configuration (4 threads, 8,844.17 ms), not the inflated $4\times$ figure from comparing against V1’s serial time or the degraded 32-thread configuration. Using the same “best deterministic CPU” standard on both platforms (Section V-A), this is somewhat larger than the GB10’s $1.46\times$, consistent with V1’s parallel scaling here being more constrained by the thread-scheduling collapse than the GB10’s CPU is.

C. Cross-Architecture Bit-Exactness

Section IV-A and Table IV each establish bit-exactness *within* a platform (CPU V1 $\equiv$ GPU V2, separately, on the GB10 and on the RTX 4090). A stronger and separate question is whether GPU V2’s output is identical *across* platforms. The GB10 (ARM host, Blackwell sm_121) and the RTX 4090 (x86 host, Ada Lovelace sm_89) differ in host instruction set architecture and GPU microarchitecture, so bit-identical output is not guaranteed by IEEE-754 double compliance alone: code generation for two different SM targets, FMA (fused multiply-add) contraction, and math-library implementations can still diverge in the last bit. We note that the two toolchains are otherwise closely matched (GCC 13.3.0 and CUDA 13.0 on both, differing only in CUDA patch level), so this test isolates the ISA and microarchitecture difference rather than compounding it with a major toolkit-version difference; a cross-toolkit determinism claim would need a separate experiment. We ran the winning 32×8 configuration on both

TABLE IV

RTX 4090: KEY CONFIGURATIONS, PERFORMANCE AND DEVIATION FROM THE VERIFIED DETERMINISTIC REFERENCE, 7-RUN REPETITIONS (MEAN $\pm$ SD OVER 6 MEASURED RUNS). FULL CPU THREAD SWEEP IN FIG. 2(B).

Variant	Configuration / Threads	Wall Time (ms)	Speedup vs. Serial	Speedup vs. Best CPU	Dev. from Ref. (dB)	SSIM	GPU L8 Kernel (ms)
CPU V0 (Naive)	1 Thread	20,973.50 $\pm$ 36.30	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (fastest)	8,207.67 $\pm$ 247.16	2.56 $\times$	N/A	68.559	0.999957	N/A
CPU V0 (Naive)	32 Threads (Max, worst)	26,388.67 $\pm$ 664.00	0.79 $\times$	N/A	23.006	0.845283	N/A
CPU V1 (Baseline)	1 Thread (Serial)	22,904.17 $\pm$ 86.90	1.00 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1 (Best)	4 Threads	8,844.17 $\pm$ 132.68	2.59 $\times$	N/A	bit-exact	bit-exact	N/A
CPU V1	32 Threads (Max, nominal)	27,411.83 $\pm$ 242.09	0.84 $\times$	N/A	bit-exact	bit-exact	N/A
GPU V2 (4t backdrop)	Grid 16 $\times$ 16_256 (baseline)	5,579.00 $\pm$ 166.61	4.11 $\times$	1.59 $\times$	bit-exact	bit-exact	437.06 $\pm$ 3.86
GPU V2 (Best Grid)	Grid 32 $\times$ 8_256	5,435.50 $\pm$ 148.01	4.21 $\times$	1.63 $\times$	bit-exact	bit-exact	360.92 $\pm$ 7.22

Speedup vs. Serial: CPU rows against their own variant’s 1-thread time; GPU V2’s denominator is CPU V1’s serial time (22,904.17 ms), since V2 has no 1-thread time of its own. Speedup vs. Best CPU: filled only for GPU V2, against CPU V1’s best configuration (4 threads, 8,844.17 ms, the fair baseline on this platform, see following subsection); remaining 3 grid configs in Fig. 3. Deviation/SSIM vs. the verified deterministic reference (Section IV-A); V1/V2 are bit-exact at every configuration tested. V0 varies non-monotonically (worst at 32 threads: 23.0 dB/0.845; the 4-thread case at 68.6 dB/0.99996 is a race that is nearly undetectable by PSNR, Section IV-A). GPU L8 Kernel times are mean $\pm$ SD over 6 runs of the dedicated profiling sweep (Section IV-E), captured separately from the wall-time sweep in this table. 32 $\times$ 8 nominally wins on both wall-time and kernel-time here, though wall-clock differences across grids are not statistically distinguishable (Section VI-D).

platforms and compared the raw output files directly: `cmp -l` reports 0 differing bytes, and the SHA-256 digests of both files are identical (8db2c07a...ca066b). GPU V2’s output is therefore bit-identical not only to its own platform’s CPU reference, but across two GPU architecture generations and two host CPU instruction set architectures, the strongest form of the determinism claim this paper makes.

D. Warp Alignment Generalizes Across GPU Architectures

At the kernel level, 32 $\times$ 8 remains fastest on this Ada Lovelace GPU (360.92 $\pm$ 7.22 ms over 6 runs), 17.4% faster than 16 $\times$ 16 (437.06 $\pm$ 3.86 ms) and 17.5% faster than the slowest configuration (437.49 $\pm$ 3.46 ms), confirming the warp-aligned coalescing argument from Section III-B (a consequence of CUDA’s 32-thread warp granularity [8]) is a property of CUDA itself, not specific to the GB10. The same localization holds here: `deconv_kernel` falls 15.6% (310.32 to 261.90 ms) while `spatial_reduction_kernel` is unchanged at 2.62 ms in both configurations, matching the GB10’s pattern on a different microarchitecture. At the *wall-clock* level the five configurations remain statistically hard to distinguish (5,435–5,579 $\pm$ 101–222 ms, overlapping error bars) even though 32 $\times$ 8 nominally wins both metrics here; the kernel-level advantage is only cleanly visible via CUDA-event profiling, reinforcing why Section IV-A’s instrumentation matters. Figure 3 compares kernel time across all five configurations on both GPUs.

E. Non-Unified Memory and FP64 Precision

This platform’s CPU/GPU memory is physically separate (64 GB DDR vs. 24 GB GDDR6X over PCIe), unlike the GB10’s unified LPDDR5x pool over NVLink-C2C [20], a distinction that recent work on unified physical memory for accelerators is beginning to study directly [21]. Despite this, GPU V2’s Layer 8 kernel (360.92 $\pm$ 7.22 ms) is faster in absolute terms than the GB10’s (533.19 $\pm$ 2.58 ms). The per-platform breakdown in Table III lets us say why, and the answer is not a single effect. Discrete-GPU PCIe/DMA

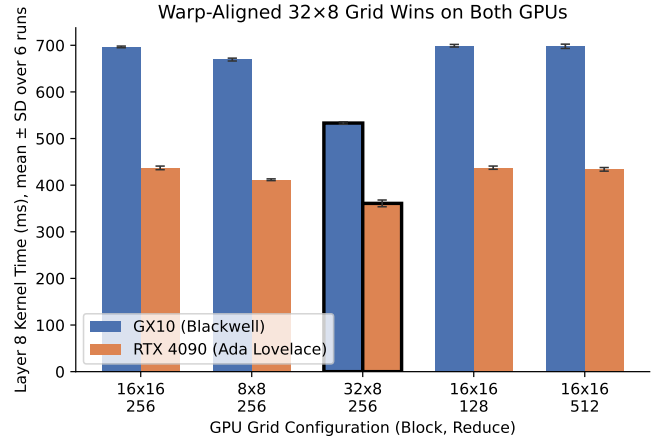


Fig. 3. GPU Layer 8 kernel execution time by grid configuration, GB10 (Blackwell) vs. RTX 4090 (Ada Lovelace). The warp-aligned 32 $\times$ 8 grid (outlined) is fastest on both GPU architectures.

overhead is well documented for transfer-bound workloads [9], and it is visible here — in-window residual is 36.5% of the RTX 4090’s event window against the GB10’s 22.1%, and D2H is 4.74 $\times$ slower — but it is outweighed twice over. The deconvolution runs 1.6 $\times$ faster on the RTX 4090, and the bandwidth-bound reduction runs 10.0 $\times$ faster because the private buffer fits in its L2 (Section IV-E). So the discrete-memory penalty is real and measurable rather than absent; it is simply smaller than the compute and cache advantages at this payload size ($\approx$ 44 MiB), and we would expect the balance to shift with a working set that overflows the RTX 4090’s L2.

The double-precision design choice (Section IV-A) has a real cost here: as a consumer GPU, the RTX 4090’s FP64 throughput is architecturally 1/64 of its FP32 throughput ($\approx$ 1.3 vs. $\approx$ 82.6 TFLOPS), so the 360.92 ms optimal kernel time (Section VI) is achieved with its dominant compute resource largely idle; an FP32 implementation would plausibly be substantially faster, at the cost of weakening bit-exactness

to a numerical-tolerance claim. The GB10’s datacenter-class GPU has a much smaller FP64:FP32 gap, so this trade-off is less severe there (not separately quantified). See Section VII for the FP32 ablation this motivates.

VII. LIMITATIONS AND FUTURE WORK

Ordered roughly by priority:

- No competitive GPU baseline: V2 is compared only against CPU implementations, not alternatives (atomicAdd accumulation, an output-stationary gather kernel, or cuDNN, Section III-B). This is now the largest gap in the evaluation.
- Cache behaviour inferred, not measured: our explanation for the $10\times$ reduction-kernel gap — that the 43.31 MiB private buffer fits the RTX 4090’s L2 but not the GB10’s (Section IV-E) — rests on effective bandwidths, one of which exceeds DRAM peak. L2 hit-rate counters (ncu) would confirm it directly, and a working-set sweep across the L2 boundary would test the design rule we draw from it.
- Warp-alignment mechanism not directly profiled: the coalescing explanation (Section III-B) is consistent with timing, and localized to `deconv_kernel` as predicted, but still unverified by profiler load-efficiency counters; a sweep including non-aligned widths (e.g. 64×4) would isolate the effect further.
- In-window transfer not separated from launch gaps: after retracting the `h2d_ms` reading (Section IV-E), our host-to-device figure is an upper bound that still contains launch overhead and host-side padding. Pinned-memory or CUDA-stream instrumentation would separate them; the D2H measurement is unaffected.
- FP64 throughout, unquantified FP32 trade-off: no FP32 ablation was run to quantify the speedup vs. bit-exactness trade-off discussed in Section VI.
- RTX 4090 collapse not isolated from confounds: no `OMP_PROC_BIND/OMP_PLACES` control experiment was run, so memory-bandwidth saturation, DVFS/thermal throttling, or false sharing remain possible alongside scheduler oversubscription.
- Reconstruction quality measured on one sequence at one scale factor (Section IV-C), with a bicubic-downsampled input standing in for a natively high-resolution source; chroma is replicated rather than reconstructed, so the pipeline’s perceptual quality is bounded by that, independent of Layer 8.
- Single test sequence/resolution: all results use the 150-frame QCIF-to-CIF Suzie sequence; higher resolutions (e.g. 1080p) would shift the compute-to-transfer ratio and, per the point above, push the private buffer past the RTX 4090’s L2.

VIII. CONCLUSION

In this paper, we introduced a GPU Spatial Reduction framework for accelerating FSRCNN’s Layer 8 deconvolution, evaluated on two architecturally distinct platforms. On both,

V1 (CPU) and V2 (GPU spatial reduction) are bit-exact against a verified deterministic reference at every configuration tested, while V0’s naive accumulation shows real, non-monotonic data-race corruption (as low as 13.8 dB/0.518 SSIM on the GB10, and a near-undetectable 68.6 dB/0.99996 on the RTX 4090; Fig. 1). On the GB10, the optimal 32×8 grid achieves $1.46\times$ speedup (31.5 FPS) over CPU V1’s best correctness-verified configuration, and $3.84\times$ over the fastest bit-exact serial run (Section V-A); the GPU Layer 8 kernels complete in 533.19 ± 2.58 ms. Cross-platform validation on a discrete-GPU desktop (Intel i9-14900K + RTX 4090, Section VI) confirms 32×8 remains fastest at the kernel level on a second GPU architecture, with GPU V2 outperforming that platform’s best CPU configuration by up to $1.63\times$ despite non-unified memory; GPU V2’s output is moreover bit-identical across both platforms directly (Section VI-C), across different host ISAs and GPU architecture generations, while also surfacing a CPU thread-scheduling pathology independent of our GPU contribution.

Per-kernel profiling on both platforms (Section IV-E) yields the paper’s sharpest architectural result and also corrected an error in our own earlier instrumentation: what we had read as a large host-to-device transfer asymmetry was a host-side counter double-counting kernel-completion waiting, and we retract it. The genuine asymmetries run in opposite directions. Unified memory returns results $4.74\times$ faster (device-to-host, 3.46 vs. 16.42 ms), while the discrete GPU runs the bandwidth-bound reduction $10.0\times$ faster because its 72 MB L2 holds the entire 43.31 MiB private buffer that the GB10 must stream from LPDDR5x at 95% of peak. The cost of determinism therefore depends less on the algorithm than on whether the privatization working set fits in last-level cache. Section VII lists the follow-up experiments needed to fully close out these findings.

REFERENCES

- [1] Annisa, Adnan, and Z. Zainuddin, “Enhancing computational efficiency: Transitioning from serial to parallel programming for low-to-high resolution video reconstruction using FSRCNN on AMP architecture,” in *Proc. IEEE Int. Conf. Artificial Intelligence and Mechatronics Systems (AIMS)*, 2025, pp. 1–8, doi: 10.1109/AIMS66189.2025.11229650.
- [2] C. Dong, C. C. Loy, and X. Tang, “Accelerating the Super-Resolution Convolutional Neural Network,” in *Proc. European Conference on Computer Vision (ECCV)*, 2016, pp. 391–407.
- [3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, “Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [5] S. Wang, G. Ananthanarayanan, Y. Zeng, N. Goel, A. Pathania, and T. Mitra, “High-Throughput CNN Inference on Embedded ARM big.LITTLE Multicore Processors,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2254–2267, Oct. 2020.
- [6] C. Bilbao, J. C. Saez, and M. Prieto-Matías, “Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake,” *Concurrency and Computation: Practice and Experience*, vol. 35, no. 25, e7814, 2023, doi: 10.1002/cpe.7814.

- [7] A. E. Eichenberger, C. Terboven, M. Wong, and D. an Mey, "The Design of OpenMP Thread Affinity," in *OpenMP in a Heterogeneous World (IWOMP 2012)*, Lecture Notes in Computer Science, vol. 7312, Springer, 2012, pp. 15–28, doi: 10.1007/978-3-642-30961-8_2.
- [8] NVIDIA Corporation, *CUDA C++ Programming Guide*, [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [9] Y. Go, M. A. Jamsheer, Y. Moon, C. Hwang, and K. Park, "APUNet: Revitalizing GPU as Packet Processing Accelerator," in *Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017, pp. 83–96.
- [10] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [11] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a Deep Convolutional Network for Image Super-Resolution," in *Proc. European Conference on Computer Vision (ECCV)*, 2014, pp. 184–199, doi: 10.1007/978-3-319-10593-2_13.
- [12] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485, doi: 10.1145/1465482.1465560.
- [13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data Race Detection in Practice," in *Proc. Workshop on Binary Instrumentation and Applications (WBIA)*, 2009, pp. 62–71, doi: 10.1145/1791194.1791203.
- [14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.
- [15] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient Primitives for Deep Learning," arXiv:1410.0759, 2014.
- [16] V. Dumoulin and F. Visin, "A Guide to Convolution Arithmetic for Deep Learning," arXiv:1603.07285, 2016.
- [17] NVIDIA Corporation, "NVIDIA Puts Grace Blackwell on Every Desk and at Every AI Developer's Fingertips," NVIDIA Newsroom, Jan. 2025. [Online]. Available: <https://nvidianews.nvidia.com/news/nvidia-puts-grace-blackwell-on-every-desk-and-at-every-ai-developers-fingertips>.
- [18] Arm Limited, "DynamIQ Technology." [Online]. Available: <https://www.arm.com/technologies/dynamiq>.
- [19] NVIDIA Corporation, *NVIDIA Ada GPU Architecture*, Whitepaper V2.02, 2022. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>.
- [20] NVIDIA Corporation, "NVIDIA Grace Hopper Superchip Architecture In-Depth," NVIDIA Developer Technical Blog, Nov. 2022. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/>.
- [21] J. Wahlgren, G. Schieffer, R. Shi, E. A. León, R. Pearce, M. Gokhale, and I. Peng, "Dissecting CPU-GPU Unified Physical Memory on AMD MI300A APUs," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2025, doi: 10.1109/IISWC66894.2025.00038.
- [22] A. Lashgar, A. Baniasadi, and A. Khonsari, "Dynamic Warp Resizing in High-Performance SIMT," arXiv:1208.2374, 2012, doi: 10.48550/arXiv.1208.2374.